SecureChain

Detect Supply-Chain Risk at the Commit Boundary

Commit-Diff Security Analysis and Multi-Model Validation

Technical Documentation

Ritvik Indupuri

Abstract

SecureChain is a containerized software supply-chain analysis platform that examines risk introduced by recent commits in public GitHub repositories. Instead of rescanning an entire codebase, it retrieves commit metadata and unified diffs, extracts newly added lines and evaluates each commit independently. Four deterministic scanners identify dependency abuse, exposed credentials, obfuscated code, suspicious network behavior and known indicators of compromise.

Only commits with deterministic evidence are forwarded to an isolated consensus service. Google Gemini produces an initial structured verdict and Anthropic Claude performs peer validation, removes unsupported findings and returns the final analyst-facing result. Clean commits bypass model review, preserving a deterministic baseline and avoiding unnecessary token use.

Document scope

- System boundaries, services and principal data flows.
- Commit acquisition, diff-scoped analysis and scan-state handling.
- Deterministic scanner coverage and evidence semantics.
- Conditional Gemini-to-Claude consensus validation.
- API, dashboard, reporting, container hardening and deployment.
- Implementation cross-reference and prioritized next steps.

Contents

Abstract 2

1 System overview 3

2 System architecture 4

3 Scan and consensus lifecycle 6

4 GitHub ingestion and commit model 8

5 Deterministic scanner layer 8

6 Multi-model consensus 10

7 Evidence, API and reporting 10

8 Container and secret boundaries 11

9 Deployment and operation 12

10 Implementation cross-reference 12

11 Next steps 13

1 System overview

SecureChain treats the commit delta as the unit of analysis. The core service walks a bounded window of recent commits from newest to oldest, analyzes only the content introduced by each change and maintains a per-commit timeline. This design makes the alert explainable: the result identifies which commit introduced evidence, which file contained it and whether the final consensus considered it worth escalating.

> *The deterministic scanners create the evidence baseline. Gemini and Claude review only flagged commits and do not replace the original scanner evidence.*

1.1 Implemented capabilities

Capability	Implementation
Commit-delta ingestion	Fetches recent public-repository commits, per-file patches and added lines through the GitHub REST API.
Parallel prescan	Runs package, secret/obfuscation, network and IOC scanners concurrently for every commit.
Conditional consensus	Routes evidence-bearing commits through Gemini and then Claude; clean commits skip model calls.
Evidence timeline	Stores scan status, repository metadata, events, per-commit evidence, verdicts and token-skip counts in memory.
Analyst delivery	Provides status and findings through REST, a browser dashboard and downloadable PDF reports.

1.2 Technology profile

Layer	Technology
Core API	Python, FastAPI, asyncio, httpx, Pydantic
Security analysis	Deterministic Python scanners and curated IOC lists
Consensus	Google Gemini 2.5 Flash and Anthropic Claude Sonnet 4 defaults
Frontend	HTML, CSS, vanilla JavaScript and D3.js
Reporting and runtime	fpdf2, Docker and Docker Compose

2 System architecture

Figure 1 separates the public repository boundary, exposed core service, internal consensus service and evidence outputs. Agent icons identify only the two model-backed review roles.

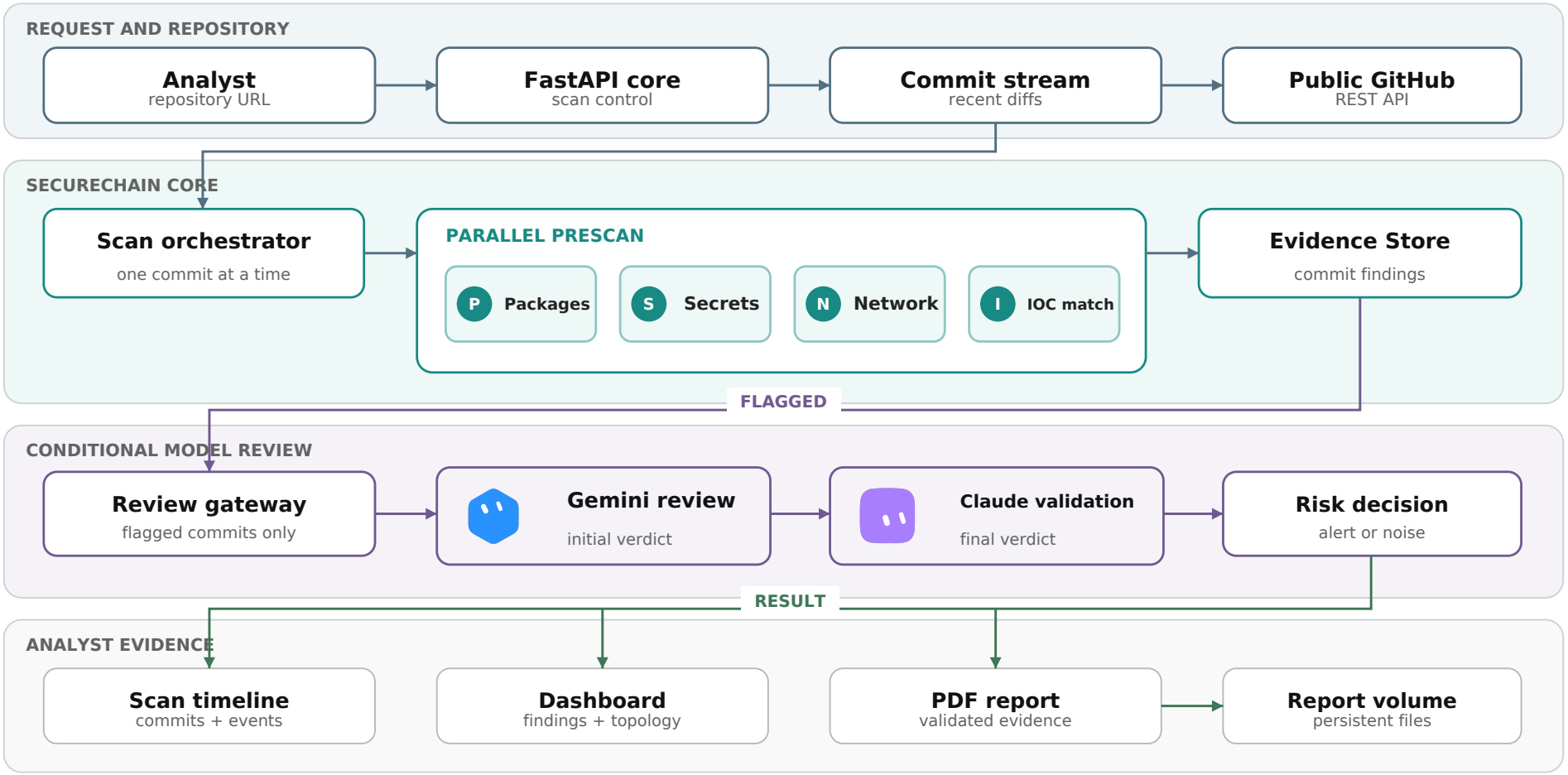


Figure 1: SecureChain logical architecture and trust boundaries.

2.1 Repository and application boundary

An analyst starts a scan through the browser interface or POST /api/scan. The exposed core container parses the repository URL, selects a recent commit window and calls the public GitHub REST API. The GitHub Core Stream retains commit metadata, changed-file attributes, patches and added lines, giving downstream scanners a diff-scoped input instead of a full repository checkout.

2.2 Core analysis plane

The core service owns orchestration, deterministic analysis, scan state and report generation. For each commit it dispatches four scanner functions concurrently with asyncio.to_thread and flattens their results into one evidence set. The Security Evidence Store records both raw evidence and any later consensus verdict while preserving clean commits in the same timeline.

2.3 Isolated consensus plane

The core container contains no model-provider keys. Flagged evidence crosses the internal Docker bridge through a narrow review endpoint. Gemini creates the first JSON verdict; Claude receives the same evidence plus Gemini's output and produces the validated result. The AI service publishes no host port, so it is reachable only from containers attached to the internal network.

2.4 Delivery and persistence

Scan status, events, severity totals, validated findings and confirmed IOCs are returned through the scan API and rendered by the analyst portal. Reports are generated on demand from completed scan state and written to the report_storage volume. The in-memory scan store is process-local; the named volume persists generated PDF files, not active scan state.

3 Scan and consensus lifecycle

Figure 2 follows one scan from repository submission through deterministic inspection, conditional model validation and analyst delivery. Each connector label appears inside an opaque gap, separate from the path it describes.

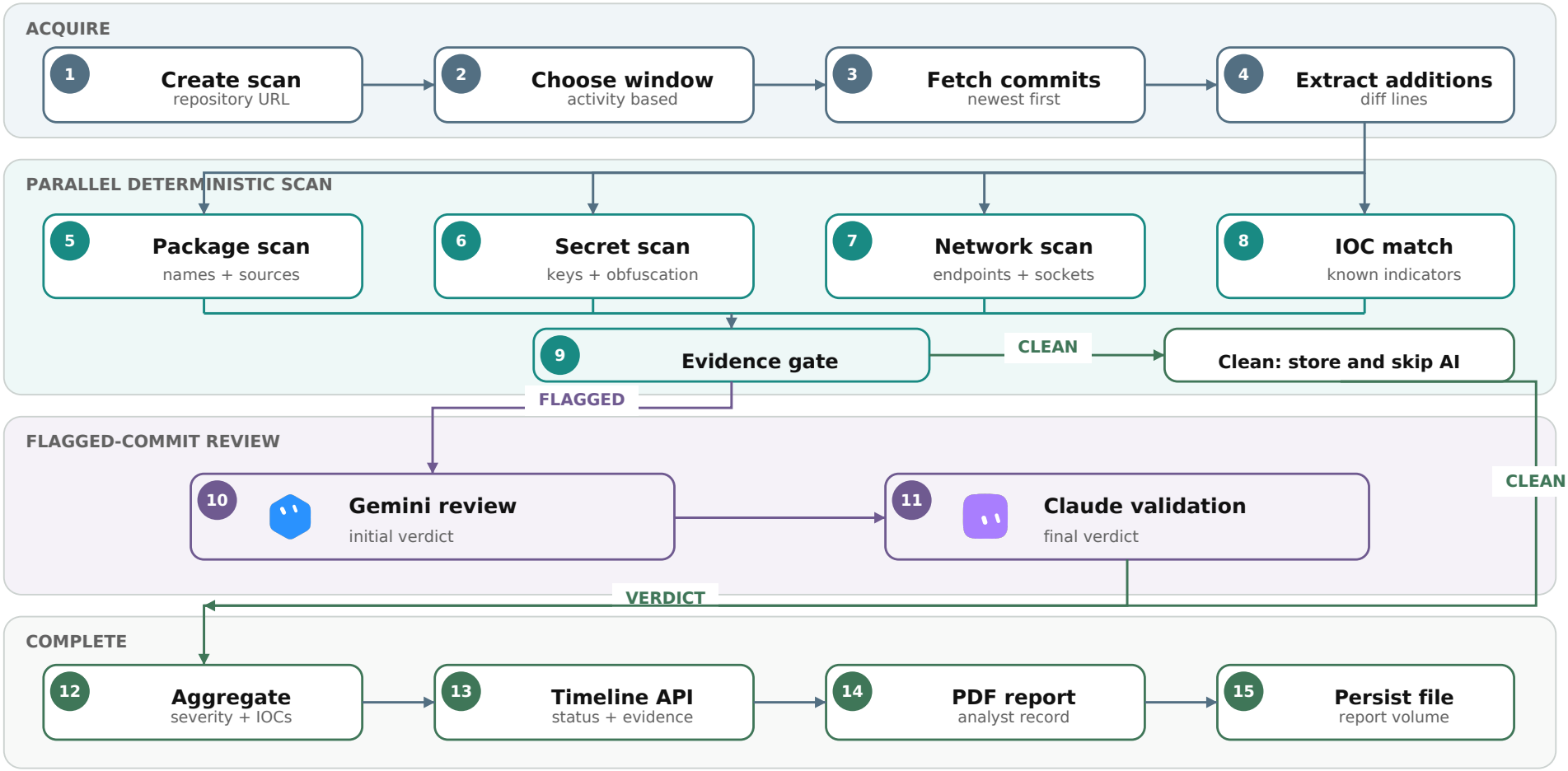


Figure 2: Per-commit analysis and conditional consensus lifecycle.

3.1 Acquisition and deterministic evaluation

A scan begins with an analyst-provided repository URL and an optional commit limit. When the limit is omitted, SecureChain samples recent activity and selects a window between three and fifteen commits; an explicit limit is clamped between one and twenty. Each fetched commit is reduced to file metadata, unified patch content and added lines before the four scanners run in parallel.

3.2 Clean and flagged branches

If no scanner emits evidence, the commit is stored as clean and model review is skipped. The scan records this decision in tokens_saved_commits. When evidence exists, the commit context and evidence list are sent to the internal consensus service. This branch preserves the low-cost deterministic pass for every commit while reserving model reasoning for potentially meaningful changes.

3.3 Consensus and scan completion

Gemini evaluates the evidence and returns an initial risk, alert decision, findings and remediation. Claude then validates that proposal, removes false positives and finalizes the structured verdict. After all commits are processed, the store aggregates severity counts, alert-worthy commits, validated findings and confirmed IOCs. The operator can then review the timeline and generate a PDF report.

4 GitHub ingestion and commit model

4.1 Repository scope

The implemented GitHub client accepts an owner/repository string or a github.com URL and is designed for public repositories. It requests repository metadata, the current default branch, a recent commit list and details for each selected commit. Private or missing repositories and API rate-limit responses are converted into explicit pipeline errors.

4.2 Diff extraction

For each changed file, SecureChain records status, category, additions, deletions, the patch and lines beginning with '+' other than the diff header. Removed files are excluded by the scanners. Filenames are classified as dependency, configuration, source or other so specialized checks can focus on relevant material.

4.3 Commit-window heuristic

Recent activity	Selected window	Rationale
20 or more commits in seven days	6 commits	Concentrate on the freshest changes.
8-19 commits in seven days	8 commits	Use the standard window.
1-7 commits in seven days	12 commits	Widen coverage for a quieter repository.
No commits in seven days	12 commits	Include older recent changes.

The scanner evaluates available GitHub patch text. Very large or binary changes may not include a patch in the GitHub commit response and therefore may not provide added-line content to the deterministic scanners.

5 Deterministic scanner layer

The four scanners operate as evidence collectors. Each finding records the producing scanner, security pillar, severity, title, explanation, filename, evidence excerpt and commit identifier. They do not call a model or external package registry.

5.1 Package and name checks

- Calculates Levenshtein distance against curated popular npm and PyPI package names to identify near-name typosquats.
- Flags direct URL, Git, local-file, archive and other non-registry dependency sources.
- Detects npm preinstall, install, postinstall and prepare hooks that can execute during package installation.
- Parses newly added dependency-manifest content, including package.json and Python requirement forms.

5.2 Secret and obfuscation checks

The integrity scanner applies high-signal patterns for cloud credentials, GitHub and Slack tokens, Google, Stripe, OpenAI and Anthropic keys, private keys, JWTs, database URIs and password assignments. It also detects encoded or intentionally obscured execution patterns, including Base64 payloads, escaped strings, dynamic evaluation and operating-system command execution.

5.3 Network and exfiltration checks

The network tracer evaluates newly added code for suspicious outbound services, public hard-coded IP addresses, raw sockets or WebSockets and combinations that read sensitive material before transmitting data. Findings preserve the source line and destination context for analyst review.

5.4 IOC matching

The IOC matcher compares commit additions with curated malicious npm and PyPI package names, commonly abused command-and-control or exfiltration hosts, high-risk top-level domains and download-and-execute or reverse-shell patterns. IOC matches remain preliminary until the consensus verdict marks a validated finding as an IOC.

> *SecureChain matches known indicators and suspicious newly introduced behavior. It does not claim to identify previously unknown vulnerabilities solely because a change was flagged.*

6 Multi-model consensus

6.1 Gemini orchestration

The isolated service converts the commit and scanner evidence into a bounded digest and asks Gemini for JSON containing commit risk, alert value, summary, IOC assessment, validated findings, explanations and remediation. The configured default is gemini-2.5-flash, and the model can be changed through GEMINI_MODEL.

6.2 Claude validation

Claude receives the same evidence together with Gemini's initial verdict. Its role is peer validation: confirm supported findings, identify benign fixtures or placeholders, remove false positives and finalize the analyst-facing JSON. The configured default is claude-sonnet-4-20250514 and can be changed through CLAUDE_MODEL.

6.3 Failure handling and authority

The core client enforces a 180-second request timeout and converts transport or service failures into an AIClientError. The pipeline records an error verdict with no alert-worthy findings and continues processing later commits. Scanner evidence remains stored even when consensus fails, allowing the analyst to distinguish detection evidence from model validation status.

Stage	Authority	Output
Deterministic scanners	Evidence baseline	Concrete commit- and file-scoped findings
Gemini	Initial reviewer	Proposed risk, findings and remediation
Claude	Peer validator	Final consensus decision and filtered findings
Analyst	Operational decision	Investigation, escalation or customer notification

7 Evidence, API and reporting

7.1 Scan state

The thread-safe EvidenceStore keys scans by a twelve-character identifier and tracks pending, streaming, scanning, reviewing, complete or error status. It also stores repository metadata, chronological events, commit results, errors and the number of clean commits that bypassed consensus.

7.2 API surface

Endpoint	Behavior
POST /api/scan	Creates a scan and starts the asynchronous pipeline.
GET /api/scan/{scan_id}	Returns status, repository data, commits, events, counts, findings and IOCs.
POST /api/scan/{scan_id}/report	Generates a report after the scan reaches complete status.
GET /api/scan/{scan_id}/report/download	Streams the generated report as application/pdf.
GET /api/health	Reports core status and AI-provider configuration health.

7.3 Analyst report

The fpdf2 report summarizes overall risk, alert recommendation, repository coverage, severity counts and confirmed IOCs. It includes a ready-to-send customer summary when alert-worthy commits exist and a newest-to-oldest finding timeline with commit, author, date, evidence, explanation and remediation. Reports are built only from current scan data.

8 Container and secret boundaries

Docker Compose defines separate core and ai-consensus services on one internal bridge network. Only the core publishes host port 8000. The frontend is mounted read-only, reports use a named volume and the model-provider keys are supplied only to the AI container.

Control	Implemented effect
Read-only root filesystem	Prevents normal writes to each container image filesystem.
cap_drop: ALL	Removes inherited Linux capabilities from both services.
no-new-privileges	Blocks privilege gain through setuid and setgid execution.
tmpfs /tmp	Provides ephemeral writable space without changing the image filesystem.
No AI host port	Restricts the consensus API to the Docker network.
Key separation	Keeps Gemini and Anthropic credentials out of the core service environment.

The application exposes the core API without an authentication layer in the current repository. Port 8000 should be bound only to an intended local or protected environment until access control and transport security are added.

9 Deployment and operation

9.1 Required configuration

The Compose stack reads GEMINI_API_KEY and ANTHROPIC_API_KEY for consensus review. GEMINI_MODEL and CLAUDE_MODEL are optional overrides. The current GitHub client uses unauthenticated public API requests; the GITHUB_TOKEN entry in .env.example is not consumed by github_stream.py.

```
docker compose up -d --build

# Analyst portal and API
http://localhost:8000

# Health check
curl http://localhost:8000/api/health
```

9.2 Operational sequence

- Submit a public GitHub repository URL and optionally specify the number of commits to inspect.
- Poll the scan endpoint or follow the dashboard event stream until the scan completes.
- Review per-commit evidence and confirm whether the consensus marked each finding worth alerting.
- Generate the PDF only after completion and retain it from the report volume as investigation evidence.
- Treat model output as review assistance; verify sensitive or customer-facing actions with an analyst.

10 Implementation cross-reference

The following map connects the documented behavior to the repository source.

Capability	Primary implementation
Scan API and orchestration	backend/main.py
GitHub activity and commit deltas	backend/github_stream.py
Per-commit evidence and aggregation	backend/evidence_store.py
Package and dependency analysis	backend/scanners/package_checker.py
Secrets and obfuscation	backend/scanners/secret_scanner.py
Network behavior analysis	backend/scanners/network_tracer.py
IOC matching and indicator sets	backend/scanners/ioc_matcher.py; backend/scanners/ioc_indicators.py
Core-to-consensus boundary	backend/ai_client.py; ai-consensus/service.py
Gemini and Claude consensus	ai-consensus/consensus.py
PDF evidence report	backend/report_generator.py
Analyst interface	frontend/index.html; frontend/app.js; frontend/style.css
Runtime isolation	docker-compose.yml; backend/Dockerfile; ai-consensus/Dockerfile

11 Next steps

Priority	Improvement	Value
1	Persist scan and evidence state	Retain timelines, verdicts and errors across core-service restarts.
2	Add authenticated GitHub access	Increase rate limits and support explicitly authorized private repositories.
3	Add API authentication and TLS	Protect scan data and report generation in shared environments.
4	Expand scanner regression tests	Lock expected evidence across manifest, secret, obfuscation, network and IOC cases.
5	Add patch-coverage telemetry	Identify commits whose GitHub response omitted analyzable patch content.
6	Separate consensus failure from benign verdict	Surface model outages as review-required rather than a non-alert decision.

11.1 Recommended delivery sequence

The first release-hardening phase should persist scan state, add authenticated GitHub access and distinguish consensus outages from benign review outcomes. These changes improve continuity and decision integrity without altering the deterministic evidence model.

The second phase should place authentication and TLS in front of the core API, then expand regression fixtures across every scanner category. Patch-coverage telemetry should be added at the same time so an analyst can see when GitHub did not return analyzable diff text for a changed file.

11.2 Completion criteria

A production-oriented release should preserve scan timelines across restarts, record explicit model-review status, authenticate every operator request and report incomplete patch coverage. Scanner tests should demonstrate stable results for clean commits, true positives, known benign fixtures and each curated IOC family.